

Reporte: Limpieza de datos ENCEVI

Erick Jahir Romo Eligio

Invalid Date

```
import pandas as pd
import numpy as np
```

Introducción

Como señala el Instituto Nacional de Estadística y Geografía (INEGI) [2], la Encuesta Nacional sobre el Consumo de Energéticos en Viviendas Particulares (ENCEVI) tuvo el objetivo de generar información estadística que permita conocer las características de consumo de distintas fuentes de energía utilizadas en las viviendas particulares y, conjuntamente, conocer sobre los hábitos y las prácticas en el manejo de energéticos.

Esta encuesta cuenta con 372 variables, de las cuales 13 son de nuestro interés. Su contenido se encuentra explicado en la siguiente sección; sin embargo, se tuvieron que desechar más del 50% de los datos debido a diversas causas.

Descripción de las variables

Para la limpieza de datos se utilizaron 13 variables: 3 proporcionadas por Hernández [1] y las otras 10 obtenidas de la ENCEVI [2], la descripción de todas ellas se encuentra en las siguientes secciones.

Consumo de electricidad en kWh por mes

El primer obstáculo con el que nos encontramos es la falta de datos relacionados con el consumo eléctrico en kWh, ya que lo único que se preguntó en la ENCEVI fue el costo que aparecía en su último recibo. A primera vista, puede parecer sencillo convertir el gasto en MXN a consumo en kWh, pero como bien lo señala [1], el gasto depende de la periodicidad, la tarifa, los impuestos, el periodo (en verano o en el resto del año), y otros gastos vinculados como penalizaciones, reconexiones, etc. Además, en muchas de las encuestas, se declaraba no conocer

alguno o todos los datos anteriores, lo que dificultaba aún más determinar el consumo en kWh. Afortunadamente, M. Hernández y D. Patiño [1] desarrollaron una metodología para estimar el consumo por periodo a partir de los datos con los que sí se cuenta y con ciertas suposiciones, consiguiendo estimar el consumo del 67.06% de los hogares encuestados.

Para calcular el consumo por mes se utilizaron dos variables de [1]. La primera es el **bill_kwh_estimation** el cual representa el consumo en kWh estimado para cada recibo, la segunda son los **bill_days** que indican cuantos días comprende cada recibo. Para calcular el consumo por mes, se dividió el consumo o **bill_kwh_estimation** entre el periodo o **bill_days** y se multiplico por 30.

Municipio

Debido a que la ENCEVI solo proporciona el estado de cada vivienda, [1] también proporcionó una estimación del municipio al que podría pertenecer cada vivienda, en la base de datos de [1], el municipio aparece como *munic_name*

Folio de la vivienda

El folio de la vivienda es un identificador único para cada vivienda, apareció en ENCEVI como **folio** y en la base de [1] como **folioviv**, pero ambos representan lo mismo. El folio es muy importante porque es una llave foránea presente en todas las tablas contenidas en la ENCEVI y en [1].

Factor de expansión

El factor de expansión aparece en la ENCEVI dentro de la tabla VIVIENDA como **factor_sem** y en la base de datos de [1] como **factor**. En ambos casos, esta variable representa el peso que se le da a cada unidad muestral para generalizar los resultados de la muestra a la población. Ya que las viviendas seleccionadas y las personas que residen en ellas representa a otras viviendas y personas de condiciones socioeconómicas similares

Estrato socioeconómico

En la ENCEVI se clasificó a las viviendas de acuerdo a las características socioeconómicas de las personas que las habitan así como características físicas y de equipamiento. Esta clasificacion es resultado de métodos estadísticos multivariados con 24 indicadores. Esta variable aparece como **est_socio** dentro de la tabla VIVIENDA y puede tomar uno de los siguientes valores:

1. Bajo
2. Medio bajo

3. Medio alto
4. Alto

Local comercial

Esta variable aparece solo en la encuesta ENCEVI dentro de la tabla ENCEVI como **local_com** y representa si la vivienda cuenta con un local comercial (tienda, peluquería, etc). Esta variable puede tomar alguno de dos valores:

1. = Si
2. = No

Electricidad de red pública

En la ENCEVI se puede diferenciar que viviendas cuentan con acceso a la red eléctrica pública mediante el uso de la variable **electri** dentro de la tabla ENCEVI, la cual puede tomar alguno de los siguientes valores:

1. = Si
2. = No

Uso de aire acondicionado

Otra variable de mucho interés es el uso de aire acondicionado, representado en la encuesta ENCEVI, dentro de la tabla ENCEVI como **uso_aire**, ya que los hogares con este recurso suelen consumir más energía de forma significativa. Esta variable puede tomar uno de dos valores:

1. = Si
2. = No

Superficie de construcción

Esta variable representa la superficie de construcción de todos los pisos de cada vivienda en metros cuadrados, se representa en la encuesta ENCEVI dentro de la tabla ENCEVI como **super_cons**, es importante diferenciarla de la superficie del terreno, ya que esta representa la huella de la casa así como las superficies no construidas (jardines, estacionamientos, etc.). La superficie de construcción puede tomar alguno de los siguientes valores:

1. = Hasta 30m²

2. = De 31 a 55m²
3. = De 56 a 75m²
4. = De 76 a 100m²
5. = De 101 a 150m²
6. = De 151 a 200m²
7. = 201 a más m²
- 8.
9. = No sabe

Número de integrantes de la vivienda

En la encuesta ENCEVI, dentro de la tabla HOGAR, el número de integrantes de cada vivienda está representado por la variable **tot_integ**. Para poder entender las diferencias entre vivienda y hogar se proporcionan los siguientes conceptos extraídos de [https://www.inegi.org.mx/app/glosario/default.html?p=ENCEVI_2018]:

- Vivienda particular: Construida o adaptada para el alojamiento de personas que forman hogares, y que en el momento del levantamiento puede estar habitada por uno o más hogares, o deshabitada.
- El hogar censal es la unidad formada por una o más personas, vinculadas o no por lazos de parentesco, que residen habitualmente en la misma vivienda particular.

Resumen de la estructura en tablas

CONSUMO [1]:

- folio “folioviv”
- Factor de expansión semestral “factor”
- Consumo en el periodo en kWh “bill_kwh_estimation”
- Periodo “bill_days”

ENCEVI

- “folio”
- local comercial “local_com”
 - 1 = Si
 - 2 = No

- Electricidad de red pública “electri”
 - 1 = Si
 - 2 = No
- Uso de aire acondicionado “uso_aire”
 - 1 = Si
 - 2 = No
- super_cons
 - 1 = Hasta 30m²
 - 2 = De 31 a 55m²
 - 3 = De 56 a 75m²
 - 4 = De 76 a 100m²
 - 5 = De 101 a 150m²
 - 6 = De 151 a 200m²
 - 7 = 201 a más m²
 - 9 = No sabe

VIVIENDA:

- “folio”
- Factor de expansión semestral “factor_sem”
- Estrato socioeconómico “est_socio”
 - 1 = Bajo
 - 2 = Medio bajo
 - 3 = Medio alto
 - 4 = Alto

HOGAR:

- “folio”
- Número de integrantes del hogar “tot_integ”

Limpieza previa

Antes de unir y limpiar los datos según los parametros que nos interesen, debemos realizar una limpieza previa para garantizar las siguientes condiciones:

- Folios repetidos: Esto no aplica para la tabla HOGAR, ya que el folio de vivienda se repite por cada hogar dentro de una misma vivienda
- Tipos esperados: Los datos tienen un tipo de variable esperado (*float, str, etc*)
- Valores esperados: No hay valores diferentes a los señalados en la descripción de las variables

```
# Importar tablas

f = "../data/ENCEVI_Hernandez/encevi_result_db_041722.csv"
CONSUMO = pd.read_csv(f,
                      usecols = ["folioviv", "factor", "bill_kwh_estimation", "bill_days", "muni"]
                      )
CONSUMO.rename(columns={"folioviv": "folio", "factor": "factor_sem"}, inplace=True)
CONSUMO.set_index('folio', inplace=True)

f = "../data/ENCEVI_raw/3_encevi_2018_base_de_datos_csv/encevi.csv"
ENCEVI = pd.read_csv(f,
                     usecols = ["folio", "super_cons", "uso_aire", 'local_com', 'electri',
                                'total_niv', 'super_cons', 'super_ter', 'tot_cuart',]
                     )
ENCEVI.set_index('folio', inplace=True)

f = "../data/ENCEVI_raw/3_encevi_2018_base_de_datos_csv/vivienda.csv"
VIVIENDA = pd.read_csv(f,
                       usecols = ["folio", "factor_sem", 'est_socio']
                       )
VIVIENDA.set_index('folio', inplace=True)

f = "../data/ENCEVI_raw/3_encevi_2018_base_de_datos_csv/hogar.csv"
HOGAR = pd.read_csv(f,
                    usecols = ["folio", "tot_integ"]
                    )
HOGAR.set_index('folio', inplace=True)
```

Folios repetidos

Como el folio es único para una vivienda, no se deben repetir para las tablas CONSUMO, ENCEVI, VIVIENDA y HOGAR

```
def revisar_duplicados_en_indice(dfs: dict):
    resultados = []

    for nombre, df in dfs.items():
        total = len(df)
        duplicados = df.index.duplicated().sum() # Cuenta entradas duplicadas en el índice
        unicos = df.index.nunique()

        resultados.append({
            "DataFrame": nombre,
            "Total registros": total,
            "FOLIO únicos": unicos,
            "Duplicados": duplicados,
        })

    return pd.DataFrame(resultados)

# Usar la función con tus DataFrames indexados por folio
dfs = {
    "CONSUMO": CONSUMO,
    "ENCEVI": ENCEVI,
    "VIVIENDA": VIVIENDA,
    "HOGAR": HOGAR
}

tabla_duplicados = revisar_duplicados_en_indice(dfs)
print(tabla_duplicados)
```

	DataFrame	Total registros	FOLIO únicos	Duplicados
0	CONSUMO	20578	20569	9
1	ENCEVI	28953	28953	0
2	VIVIENDA	28953	28953	0
3	HOGAR	28953	28953	0

```
# Identificar los duplicados en el índice
duplicados_mask = CONSUMO.index.duplicated(keep=False)
```

```

# Separar duplicados y únicos
duplicados = CONSUMO[duplicados_mask]
unicos = CONSUMO[~duplicados_mask]

# Sumar los valores de 'factor_sum' de los duplicados
Tirado_CONSUMO_folio_duplicado = duplicados['factor_sem'].sum()

# Reemplazar CONSUMO con solo los registros únicos
CONSUMO = unicos

# Mostrar la suma "tirada"
print(f"Numero de viviendas tiradas por tener folio duplicado en CONSUMO: {Tirado_CONSUMO_fo

```

Numero de viviendas tiradas por tener folio duplicado en CONSUMO: 25,688

Tipos de valores esperados

Lo que se espera es que todas las variables sean *int* o *float* menos **munic_name** que sería *object* Lo que podemos ver en la tabla inferior es que las variables **uso_aire** y **local_com** no cumplen con esta condición.

```

# Lista de tus DataFrames con sus nombres
dfs = {
    "CONSUMO": CONSUMO,
    "ENCEVI": ENCEVI,
    "VIVIENDA": VIVIENDA,
    "HOGAR": HOGAR
}

# Creamos una lista para almacenar los resultados
info = []

# Recorremos cada DataFrame
for nombre, df in dfs.items():
    for col in df.columns:
        info.append({
            "DataFrame": nombre,
            "Columna": col,
            "Tipo de dato": df[col].dtype
        })

```



```
# Convertimos a DataFrame
tabla_tipos = pd.DataFrame(info)

# Mostramos la tabla ordenada por DataFrame
print(tabla_tipos.sort_values(by="DataFrame"))
```

	DataFrame	Columna	Tipo de dato
0	CONSUMO	factor_sem	int64
1	CONSUMO	bill_days	int64
2	CONSUMO	munic_name	object
3	CONSUMO	bill_kwh_estimation	float64
4	ENCEVI	total_niv	int64
5	ENCEVI	super_ter	int64
6	ENCEVI	super_cons	int64
7	ENCEVI	tot_cuart	int64
8	ENCEVI	electri	int64
9	ENCEVI	local_com	object
10	ENCEVI	uso_aire	object
13	HOGAR	tot_integ	int64
11	VIVIENDA	est_socio	int64
12	VIVIENDA	factor_sem	int64

```
columnas = ENCEVI.select_dtypes(include='object').columns

for col in columnas:
    # Reemplazar cadenas vacías o espacios con NaN
    ENCEVI[col] = ENCEVI[col].replace(r'^\s*$', np.nan, regex=True)

    # Intentar convertir a numérico (NaN si falla)
    ENCEVI[col] = pd.to_numeric(ENCEVI[col], errors='coerce')

    # Convertir a tipo entero que soporte NaN
    ENCEVI[col] = ENCEVI[col].astype('Int64')
```

Ahora ya se cumple la primera condicion como se puede ver abajo.

```
# Lista de tus DataFrames con sus nombres
dfs = {
    "CONSUMO": CONSUMO,
    "ENCEVI": ENCEVI,
    "VIVIENDA": VIVIENDA,
```

```

    "HOGAR": HOGAR
}

# Creamos una lista para almacenar los resultados
info = []

# Recorremos cada DataFrame
for nombre, df in dfs.items():
    for col in df.columns:
        info.append({
            "DataFrame": nombre,
            "Columna": col,
            "Tipo de dato": df[col].dtype
        })

# Convertimos a DataFrame
tabla_tipos = pd.DataFrame(info)

# Mostramos la tabla ordenada por DataFrame
print(tabla_tipos.sort_values(by="DataFrame"))

```

	DataFrame	Columna	Tipo de dato
0	CONSUMO	factor_sem	int64
1	CONSUMO	bill_days	int64
2	CONSUMO	munic_name	object
3	CONSUMO	bill_kwh_estimation	float64
4	ENCEVI	total_niv	int64
5	ENCEVI	super_ter	int64
6	ENCEVI	super_cons	int64
7	ENCEVI	tot_cuart	int64
8	ENCEVI	electri	int64
9	ENCEVI	local_com	Int64
10	ENCEVI	uso_aire	Int64
13	HOGAR	tot_integ	int64
11	VIVIENDA	est_socio	int64
12	VIVIENDA	factor_sem	int64

Valores esperados

Por distintas razones como pérdida de datos, errores al anotar o capturar, se puede llegar a tener valores diferentes a los que nos esperamos en la descripción de variables. Las variables

que tienen valores fijos esperados son:

- local comercial “local_com”
 - 1 = Si
 - 2 = No
- Electricidad de red pública “electri”
 - 1 = Si
 - 2 = No
- Uso de aire acondicionado “uso_aire”
 - 1 = Si
 - 2 = No
- super_cons
 - 1 = Hasta 30m²
 - 2 = De 31 a 55m²
 - 3 = De 56 a 75m²
 - 4 = De 76 a 100m²
 - 5 = De 101 a 150m²
 - 6 = De 151 a 200m²
 - 7 = 201 a más m²
 - 9 = No sabe
- Estrato socioeconómico “est_socio”
 - 1 = Bajo
 - 2 = Medio bajo
 - 3 = Medio alto
 - 4 = Alto

Como podemos ver, las columnas local_com y uso_aire tienen valores NaN que tendrán que ser descartados más adelante.

```

import pandas as pd

columnas = [
    "local_com",
    "electri",
    "uso_aire",
    "super_cons",
    "est_socio"
]

dataframes = [
    ENCEVI,
    ENCEVI,
    ENCEVI,
    ENCEVI,
    VIVIENDA
]

# Lista para guardar los resultados
resultados = []

for col, df in zip(columnas, dataframes):
    conteo = df[col].value_counts(dropna=False)
    porcentaje = df[col].value_counts(normalize=True, dropna=False) * 100

    # Convertir a DataFrame temporal
    temp_df = pd.DataFrame({
        "variable": col,
        "valor": conteo.index,
        "frecuencia": conteo.values,
        "porcentaje": porcentaje.values.round(2)
    })

    resultados.append(temp_df)

# Unir todos los resultados en un solo DataFrame
tabla_resultado = pd.concat(resultados, ignore_index=True)

# Mostrar
print(tabla_resultado)

```

```

variable  valor  frecuencia  porcentaje

```

0	local_com	2	26830	92.67
1	local_com	1	1847	6.38
2	local_com	<NA>	276	0.95
3	electri	1	28677	99.05
4	electri	2	276	0.95
5	uso_aire	2	23544	81.32
6	uso_aire	1	5225	18.05
7	uso_aire	<NA>	184	0.64
8	super_cons	9	6502	22.46
9	super_cons	4	4977	17.19
10	super_cons	3	4519	15.61
11	super_cons	2	4008	13.84
12	super_cons	5	3735	12.9
13	super_cons	1	2148	7.42
14	super_cons	6	1828	6.31
15	super_cons	7	1236	4.27
16	est_socio	2	15126	52.24
17	est_socio	3	5884	20.32
18	est_socio	1	5505	19.01
19	est_socio	4	2438	8.42

Unir las tablas y limpieza

Para poder continuar con el procesamiento y limpieza de los datos es necesario unir las tablas para poder acceder al factor de expansión y contabilizar cuántos datos se van tirando en cada paso. El proceso de limpieza tiene el objetivo de discriminar los datos mediante los siguientes parametros

- Valores sin NaN
- Viviendas con conexión a la red pública
- Viviendas sin negocio
- Por tamaño de vivienda

Unir tablas ENCEVI, VIVIENDA y HOGAR

Es fácil unirlos ya que todos tienen el mismo número de datos

```
encuesta_ENCEVI = pd.concat([ENCEVI, VIVIENDA, HOGAR], axis=1)
encuesta_ENCEVI.head(5)
```

	total_niv	super_ter	super_cons	tot_cuart	electri	local_com	uso_aire	est_socio	factor_s
folio									
1	2	3	7	11	1	2	2	3	373
2	1	3	3	6	1	2	2	3	373
3	2	3	7	5	1	2	2	3	373
4	1	3	4	6	1	2	2	3	373
5	1	3	4	5	1	2	2	3	373

```
VIVIENDA_original = VIVIENDA.copy()
Total_viviendas_ENCEVI = VIVIENDA_original.factor_sem.sum()

print(f"En la encuesta ENCEVI sin limpiar tenemos {Total_viviendas_ENCEVI:,.0f} viviendas ")
```

En la encuesta ENCEVI sin limpiar tenemos 33,162,148 viviendas

Tirar datos por tener NaN

No nos interesan los valores que se hayan perdido u omitido, por esa razón se tiran los datos con valores NaN

```
encuesta_ENCEVI_original = encuesta_ENCEVI.copy()
encuesta_ENCEVI = encuesta_ENCEVI.dropna()

tirados_NaN = Total_viviendas_ENCEVI - encuesta_ENCEVI.factor_sem.sum()
tirados_NaN_porcentaje = (tirados_NaN *100 / Total_viviendas_ENCEVI)

print(f"Se tiraron {tirados_NaN:,.0f} viviendas del total original, que equivale al {round(tirados_NaN_porcentaje, 2)}%")
print(f"Nos quedamos con {encuesta_ENCEVI.factor_sem.sum():,.0f} viviendas")
```

Se tiraron 353,640 viviendas del total original, que equivale al 1.07%
Nos quedamos con 32,808,508 viviendas

```
CONSUMO_orignal = CONSUMO.copy()
CONSUMO = CONSUMO.dropna()

tirados_NaN_CONSUMO = CONSUMO_orignal.factor_sem.sum() - CONSUMO.factor_sem.sum()
tirados_NaN_CONSUMO_porcentaje = (tirados_NaN_CONSUMO*100)/Total_viviendas_ENCEVI

print(f"Se tiraron {tirados_NaN_CONSUMO:,.0f} viviendas de la base de datos de Hernandez por {tirados_NaN_CONSUMO_porcentaje}%")
print(f"Nos quedamos con {CONSUMO.factor_sem.sum():,.0f} viviendas de HERNANDEZ [1]")
```

Se tiraron 74,034 viviendas de la base de datos de Hernandez por tener NaN, que equivale al 1.07%.
Nos quedamos con 22,237,360 viviendas de HERNANDEZ [1]

```
CONSUMO.columns
```

```
Index(['factor_sem', 'bill_days', 'munic_name', 'bill_kwh_estimation'], dtype='object')
```

Tirar viviendas sin conexión a la red pública

Como nos interesa analizar el consumo de energía eléctrica, no nos es útil los datos de las viviendas que están desconectadas de la red pública.

```
# Calcular la suma de 'factor_sem' donde 'electri' es 2
tirados_no_luz = encuesta_ENCEVI_original.loc[encuesta_ENCEVI_original['electri'] == 2, 'factor_sem']

# Eliminar las filas donde 'electri' es 2
encuesta_ENCEVI = encuesta_ENCEVI[encuesta_ENCEVI['electri'] != 2]

tirados_no_luz_pocentaje = (tirados_no_luz * 100 / Total_viviendas_ENCEVI)

print(f"Se tiraron {tirados_no_luz.sum():.0f} viviendas del total original, que equivale al {round(tirados_no_luz_pocentaje, 2)}%")
print(f"Nos quedamos con {encuesta_ENCEVI.factor_sem.sum():.0f} viviendas")
```

Se tiraron 353,640 viviendas del total original, que equivale al 1.07%.
Nos quedamos con 32,808,508 viviendas

Curiosamente, coincidió que todos los que habían contestado que no tenían conexión a la red pública, también tenían un valor NaN en alguna otra variable.

```
# Filtrar filas donde electri == 2
filas_electri_2 = encuesta_ENCEVI_original[encuesta_ENCEVI_original['electri'] == 2]

# Verificar si todas esas filas tienen al menos un NaN en alguna columna
todas_tienen_NaN = filas_electri_2.isnull().any(axis=1).all()

print("¿Todas las filas con sin conexión a la red pública tienen al menos un NaN?:", todas_tienen_NaN)
```

```
¿Todas las filas con sin conexión a la red pública tienen al menos un NaN?: True
```

Tirar datos por tener un local comercial

Debido a que el consumo de los aparatos relacionados con los locales comerciales aumentan el consumo de una vivienda, no se considerarán las que tengan uno.

```
# Calcular la suma de 'factor_sem' donde 'electri' es 2
tirados_local_com = encuesta_ENCEVI_original.loc[encuesta_ENCEVI_original['local_com'] == 1,

# Eliminar las filas donde 'electri' es 2
encuesta_ENCEVI = encuesta_ENCEVI[encuesta_ENCEVI['local_com'] != 1]

tirados_local_com_pocentaje = (tirados_local_com *100 / Total_viviendas_ENCEVI)

print(f"Se tiraron {tirados_local_com:,.0f} viviendas del total original, que equivale al {r
print(f"Nos quedamos con {encuesta_ENCEVI.factor_sem.sum():,.0f} viviendas")
```

Se tiraron 2,046,003 viviendas del total original, que equivale al 6.17%
Nos quedamos con 30,762,505 viviendas

Por tamaño de vivienda

Solo nos interesan las viviendas que hayan contestado que conocen el tamaño de sus viviendas y que es menor a 200 m2. Ya que para las viviendas mayores a 201 m2, no tiene sentido relacionar el consumo con el tamaño, ya que el rango de 201 a infinito es muy grande.

```
# Calcular la suma de 'factor_sem' donde 'electri' es 2
tirados_tamano = encuesta_ENCEVI_original.loc[
    encuesta_ENCEVI_original['super_cons'].isin([7,9]),
    'factor_sem'
].sum()
# Eliminar las filas donde 'electri' es 2
encuesta_ENCEVI = encuesta_ENCEVI[~encuesta_ENCEVI['super_cons'].isin([7,9])]

tirados_tamano_pocentaje = (tirados_tamano *100 / Total_viviendas_ENCEVI)

print(f"Se tiraron {tirados_tamano:,.0f} viviendas del total original, que equivale al {round
print(f"Nos quedamos con {encuesta_ENCEVI.factor_sem.sum():,.0f} viviendas")
```

Se tiraron 8,966,153 viviendas del total original, que equivale al 27.04%
Nos quedamos con 22,454,319 viviendas

Unir tablas encuesta_ENCEVI y CONSUMO

En este caso hay que unir las tablas utilizando el folio que tienen por índice. Se perderán datos ya que no todos los que quedan en ENCEVI tienen un consumo estimado por Hernandez.

```
CONSUMO_ = CONSUMO.drop('factor_sem', axis=1).copy()
ENCEVI_CONSUMO = CONSUMO_.join(encuesta_ENCEVI, how='inner')

tirados_sin_consumo = Total_viviendas_ENCEVI - CONSUMO.factor_sem.sum()

tirados_sin_consumo_porcentaje = (tirados_sin_consumo *100) / Total_viviendas_ENCEVI

print(f"Se tiraron {tirados_sin_consumo:,.0f} viviendas del total original, que equivale al .")
print(f"Nos quedamos con {ENCEVI_CONSUMO.factor_sem.sum():,.0f} viviendas")
```

Se tiraron 10,924,788 viviendas del total original, que equivale al 32.94%
Nos quedamos con 15,671,554 viviendas

Analisis de resultados y conclusiones

Como podemos observar en la tabla inferior, los criterios que más datos descartaron fueron la falta de un consumo estimado por Hernandez [1], y por tener una superficie de construcción desconocida o mayor a 201 m2. También cabe destacar, que al aplicar todos los criterios juntos, se descarta un porcentaje menor que la suma de los porcentajes por cada criterio, esto se debe a que un mismo dato descartado puede cumplir con mas de un criterio, como vimos antes, por ejemplo, todas las viviendas que no tenían conexión a la red pública, también contaban con almenos un NaN, esto quiere decir que al descartar utilizando esos dos criterios, se pierde solo el 1.07% en lugar de 2.14% que sería la suma de ambos criterios.

En conclusión, al aplicar todos los criterios se descartó más de la mitad de los datos que se tenían en un inicio. A partir de este hecho hay que tomar decisiones sobre qué análisis se le dará a este grupo de datos, por ejemplo, determinar si se descartaron más datos de un grupo en particular, por ejemplo, que se hayan descartado más datos del estrato socioeconómico bajo, o de uno o varios bioclimas en particular.

```
resumen = []

# Filtro 1: Eliminar NaN

resumen.append({
    "Criterio": "Valores NaN",
    "Viviendas perdidas": tirados_NaN,
```

```

    "Porcentaje (%)": round(tirados_NaN_pocentaje,2)
})
resumen.append({
    "Criterio": "Sin conexión a la red pública",
    "Viviendas perdidas": tirados_no_luz ,
    "Porcentaje (%)": round(tirados_no_luz_pocentaje,2)
})
resumen.append({
    "Criterio": "Con un local comercial",
    "Viviendas perdidas": tirados_local_com,
    "Porcentaje (%)": round(tirados_local_com_pocentaje ,2)
})

resumen.append({
    "Criterio": "Por superficie de construcción",
    "Viviendas perdidas": tirados_tamano ,
    "Porcentaje (%)": round(tirados_tamano_pocentaje ,2)
})
resumen.append({
    "Criterio": "Sin consumo",
    "Viviendas perdidas": tirados_sin_consumo ,
    "Porcentaje (%)": round(tirados_sin_consumo_porcentaje ,2)
})

tirados_total = Total_viviendas_ENCEVI - ENCEVI_CONSUMO.factor_sem.sum()

tirados_total_porcentaje = (tirados_total *100) / Total_viviendas_ENCEVI

resumen.append({
    "Criterio": "Todos los criterios juntos",
    "Viviendas perdidas": tirados_total ,
    "Porcentaje (%)": round(tirados_total_porcentaje ,2)
})

resumen_df = pd.DataFrame(resumen)

# Mostrar resumen
print(resumen_df)

```

	Criterio	Viviendas perdidas	Porcentaje (%)
0	Valores NaN	353640	1.07
1	Sin conexión a la red pública	353640	1.07

2	Con un local comercial	2046003	6.17
3	Por superficie de construcción	8966153	27.04
4	Sin consumo	10924788	32.94
5	Todos los criterios juntos	17490594	52.74

```
ENCEVI_CONSUMO.to_parquet("../data/modificado/ENCEVI_CONSUMO.parquet", index=False)
```

Bibliografía

- [1] M. Hernandez and D. Patino-Echeverri, “Electricity consumption, subsidies, and policy inequalities in Mexico: Data from 100,000 households,” *Energy for Sustainable Development*, vol. 67, pp. 232–242, 2022. [En línea]. Disponible en: <https://www.scopus.com/record/display.uri?eid=2-s2.0-85138759242>
- [2] Instituto Nacional de Estadística y Geografía (INEGI), Encuesta Nacional sobre Consumo de Energéticos en Viviendas Particulares 2018 (ENCEVI 2018), México, 2018. [En línea]. Disponible en: <https://www.inegi.org.mx/rnm/index.php/catalog/495>